

CHAVALES ENGINE

Documento sobre la arquitectura de la aplicación y el proceso de desarrollo

Andrés García, Ismael Ortega, Cynthia Tristán, Pablo Iglesias, Iván Palomino, Sergio Naranjo, Jule Page, Nieves Alonso, Daniel Ramos y Javier Gómez

CHAVALES

Estructura de la solución.....	1
Estructura de proyectos.....	1
Relación motor-juego:.....	2
BUILD:.....	2
Estructura de los repositorios de Git.....	2
Repositorios:.....	2
Repositorio motor:.....	3
Repositorio de cada juego (ejemplo):.....	4
Arquitectura del motor y estructura de las clases.....	5
Capa Core.....	5
Capa Módulos del motor.....	5
Capa ejecutable.....	6
Capa de gameplay.....	6
Estructura de componentes del motor.....	7
Estructura de componentes de MATABICHOS.....	8
Estructura de componentes de TIRABOLAS.....	8
Pipeline de generación de contenido.....	9

Estructura de la solución

1) Carpeta src: carpeta donde se encontrar los archivos .h y .cpp de cada módulo del motor; hay dos carpetas para los dos tipos de archivos:

- Render
 - cpp (archivos .cpp)
 - include (archivos .h)
- Físicas
 - cpp (archivos .cpp)
 - include (archivos .h)
- Audio
 - cpp (archivos .cpp)
 - include (archivos .h)
- Input
 - cpp (archivos .cpp)

- include (archivos .h)
- UI
 - cpp (archivos .cpp)
 - include (archivos .h)
- Main: referencia a los demás módulos para poder generar el ejecutable
- main.cpp

2) Carpeta bin: carpeta donde se guardarán los .exe + .dll generados al compilar propios del motor de cada configuración, en función de la versión del dispositivo se usará una versión u otra. La carpeta intermedia “Media”, sirve para meter los recursos necesarios como:

- x64
 - Debug
 - Media
 - Release
 - Media
- x86
 - Debug
 - Media
 - Release
 - Media

3) Carpeta tmp: crea archivos binarios temporales que se eliminan al cerrar al cerrar el programa.

- x64
 - Debug
 - Release
- x86
 - Debug
 - Release

4) Carpeta libraries: carpeta donde se generarán los .dll y .lib de cada módulo; varias aplicaciones (juegos) pueden compartir la misma librería y permite modificaciones en la librería sin necesidad de recompilar el ejecutable; sin embargo, es necesario que el usuario tenga las .dll instaladas y se enlazan durante ejecución; es decir no se incorpora al .exe(ejecutable).

- Render
- Físicas
- Audio
- Input
- UI

5) Carpeta projects: carpeta donde se encuentra cada proyecto del módulo del motor.

- Render
- Físicas
- Audio
- Input
- UI

- 6) Carpeta .bats: carpeta donde se guardarán archivos .bat, que facilitarán compilación u otras funcionalidades para el desarrollo del proyecto.
- 7) Carpeta para lua: se guardará todo el scripting relacionado con lua.

Estructura de proyectos

Estructura del motor:

- **MotorCore** (Static Library): GameLoop, gestión escenas, EC... Es el núcleo del motor.
- **MotorModules** (Static Library): RenderModule(Ogre), PhysxModule(PhysX), AudioModule (inicialización dependencias).
- **MotorAPI** (Static Library) (interfaz pública): Une core, system y modules para el programador.
- **MotorExe** (Executable): Proyecto que lanzas, solo incluye main.cpp, no contiene lógica de gameplay.

El motor se divide en varios proyectos independientes para separar responsabilidades. Además cada juego es independiente del motor y éstos se cargan dinámicamente.

Estructura Juego_Tirabolas

|
|—TirabolasGame (DLL): lógica del juego, gameplay, niveles...
|—TirabolasData (Resources / Data only): assets, texturas, scripts...

Estructura Juego_Matabichos

|
|—MatabichosGame (DLL): lógica del juego, gameplay, niveles...
|—MatabichosData (Resources / Data only): assets, texturas, scripts...

Relación motor-juego:

MotorExe

|
|— carga dinámicamente -> TirabolasGame.dll
|— carga dinámicamente -> MatabichosGame.dll

BUILD:

Build/
| bin/
| |— MotorLauncher.exe
| |— OgreMain.dll
| |— Render_DirectX.dll
| |— Tirabolas.dll (ej)
| |— scripts/ (LUA)
| |— Assets/

Estructura de los repositorios de Git

Repositorios:

Motor
Juego_Tirabolas
Juego_Matabichos

Repositorio motor:

```
Motor/  
|---- README.md  
|---- Engine/ (proyecto)  
|      |---- Core/ (contenido)  
|      |      |---- Debug/  
|      |      |---- Timer/  
|      |      |---- Math/ (Vectors, Quaternions, etc.)  
|      |      |---- Colors/  
|      |      |---- Entity/  
|      |      |---- Component/  
|      |      |---- Scene/  
|      |  
|      |---- SystemLayer/ (contenido)  
|      |      |---- GameLoop/  
|      |      |---- SceneManager/  
|      |  
|      |---- Plugins/ (contenido)  
|      |      |---- PluginManager/  
|      |  
|      |---- Tools/ (contenido)  
|      |      |---- Editor/  
|      |      |---- ParticleEditor/  
|      |      |---- Optimization/  
|  
|---- Modules/  
|      |---- Render/ (proyecto)  
|      |      |---- LowLevel/  
|      |      |---- Frontend/  
|      |  
|      |---- Physics/ (proyecto)  
|      |  
|      |---- Audio/ (proyecto)  
|      |  
|      |---- Resources/ (proyecto)
```

```

|
|
|---- Platform/ (proyecto)
|        |---- Input/
|        |----Window/
|
|---- Dependencies/ (proyecto)
|        |---- Ogre/
|        |---- PhysX/
|        |---- FMOD/
|        |---- Lua/
|
|---- Build/ (proyecto)
|        |---- CMake/
|        |---- Scripts/
|
|---- Docs/
|        |---- Architecture/

```

Repositorio de cada juego (ejemplo):

```

Juego_Tirabolas/
|---- README.md
|---- Game/
|        |---- Components/
|        |---- GameLogic/
|
|---- Plugins/
|        |---- TirabolasPlugin/
|
|---- Data/
|        |---- Maps/
|        |---- Blueprints/
|        |---- Config/
|
|---- Scripts/
|        |---- Lua/
|
|---- Assets/
|        |---- Models/
|        |---- Textures/
|        |---- Audio/
|
|---- Build/
|        |---- CMake/
|
|---- Docs/

```

|---- GDD/

Arquitectura del motor y estructura de las clases

Hemos decidido separar el CHAVALES Engine en diferentes capas, pasando desde las capas más fundamentales, hasta las interfaces ofrecidas al usuario.

Capa Core.

La capa core es el pilar fundamental sobre el que se cimienta el motor, contiene definiciones y sistemas esenciales que el resto del motor necesita para existir. Se proporciona una **interfaz común** sin depender de ninguna de las capas superiores. se compilará como una librería estática

Algunas de las definiciones contempladas para esta capa son las siguientes:

- **Sistema de Tiempo (Timer).** Encargado de hablar con el hardware para medir microsegundos, gestiona pausas, escalas de tiempo, y le da las herramientas a la state machine para calcular el deltatime.
- **Sistema de Logging y Debug.** Sistema generalizado para salida de datos, canaliza errores hacia la consola o un archivo, provee de etiquetas (Info, Warning, Error).
- **Datos primitivos.**
 - **Matemáticas.** Aquí se encuentran estructuras y cálculos para vectores (Vec2, Vec3, Vec4), matrices, y cuaterniones.
 - **Colores.** Estructura de 4 bytes RGBA, con cálculos para modificaciones a colores.
- **Patrón EC:**
 - **Entidad.** Clase contenedora de componentes con un identificador (uint32_t)
 - **IComponent.** Interfaz (clase abstracta) de un componente, de esta heredan todos los componentes, contiene métodos virtuales como "init()", "update()", "onEnable()", "onDestroy"...
 - **Eventos.** Definiciones de "mensajes" que pueden viajar por el sistema de eventos para todas las entidades (Ej. "CollisionEvent").

Capa Módulos del motor

La capa de módulos de motor construye el núcleo de librerías estáticas que abstrae el hardware y bibliotecas externas (SDL, Ogre, PhysX, FMOD...).

Se busca ofrecer una interfaz limpia y segura a través de una **clase estática (MotorAPI)** para que puedan interactuar las capas superiores sin acceder directamente al software de bajo nivel.

Los diferentes módulos del motor son:

- **Módulo de Plataforma (SDL3):** Actúa como una abstracción de la plataforma, en nuestro caso solo tenemos 1 plataforma, pero lo dejamos preparado en caso de querer añadir más.
 - **Ventana.** Gestiona el ciclo de vida de la ventana y provee el contexto necesario para el módulo de renderizado.
 - **Input.** Abstrae el manejo de periféricos a través de "dispositivos virtuales" permitiendo gestionar múltiples jugadores y mapeos de controles de forma unificada.
- **Módulo de Recursos:** Es el encargado de la gestión de recursos, Su objetivo es garantizar la eficiencia de la ram, por ejemplo, si dos entidades requieren el mismo recurso, lo carga solo una vez y se comparte.

- **Módulo de Render.** Provee de una interfaz que engloba librerías gráficas, se subdivide funcionalmente en.
 - **LowLevel (Ogre).** Encapsulado directo de renderizado de mallas, sprites, luces, shaders, partículas...
 - **FrontEnd (RmlUI).** Sistema dedicado a la creación de interfaces de usuario utilizando archivos html y css.
 - **Optimización.** Herramientas para la mejora de rendimiento, como sistemas LOD y Culling.
- **Módulo de física (PhysX).** Encapsula el motor PhysX para gestionar escenas físicas internas y sincronizarlas con entidades de juego.
- **Módulo de audio (FMOD).** Gestiona el hardware de sonido y la mezcla mediante FMOD. Además permite el control de canales y permite aplicación de efectos.

Capa ejecutable

Esta capa es la encargada de inicializar el motor, gestionar el ciclo de vida de la aplicación y servir de puente para la lógica externa.

- **Main().** Es el disparador de toda la infraestructura.
 - Inicializa sistemas. `MotorAPI::init()`
 - Arranque de Managers. Instancia controladores críticos `DLLoader` y `StateMachine`.
- **DLLoader.** Gestor de enlaces dinámicos, es el puente que enlaza componentes y lógica del usuario desarrollador (Game y Plugins). Expone también funciones para cargar y descargar librerías.
- **StateMachine.** Contiene el bucle de juego, gestión de de escenas y calcula el `deltaTime` entre frames.
- **GameLoader.** Interpreta los datos de juego.
 - Lee los scripts de LUA para instanciar escenas, entidades, componentes y prefabs definidos en los scripts, y utiliza los componentes proveídos por la DLL del juego/Plugin

Capa de gameplay

En esta capa se contiene el contenido específico de cada juego, El desarrollador se encarga de programar **componentes en C++** con los que se generará una librería dinámica, y utilizará LUA para definir escenas, con las entidades que contiene y los componentes que componen las entidades. además de especificar plugins en caso de ser necesario.

Los **plugins** en nuestro motor funcionan como un juego, y añaden funcionalidades extra a los componentes ya definidos.

Por último nos gustaría implementar **tools** que ayuden a desarrollar al usuario, aunque lo planteamos como un **nice to have**, el objetivo es abstraer complejidad técnica para que diseñadores puedan configurar escenas y efectos de manera visual.

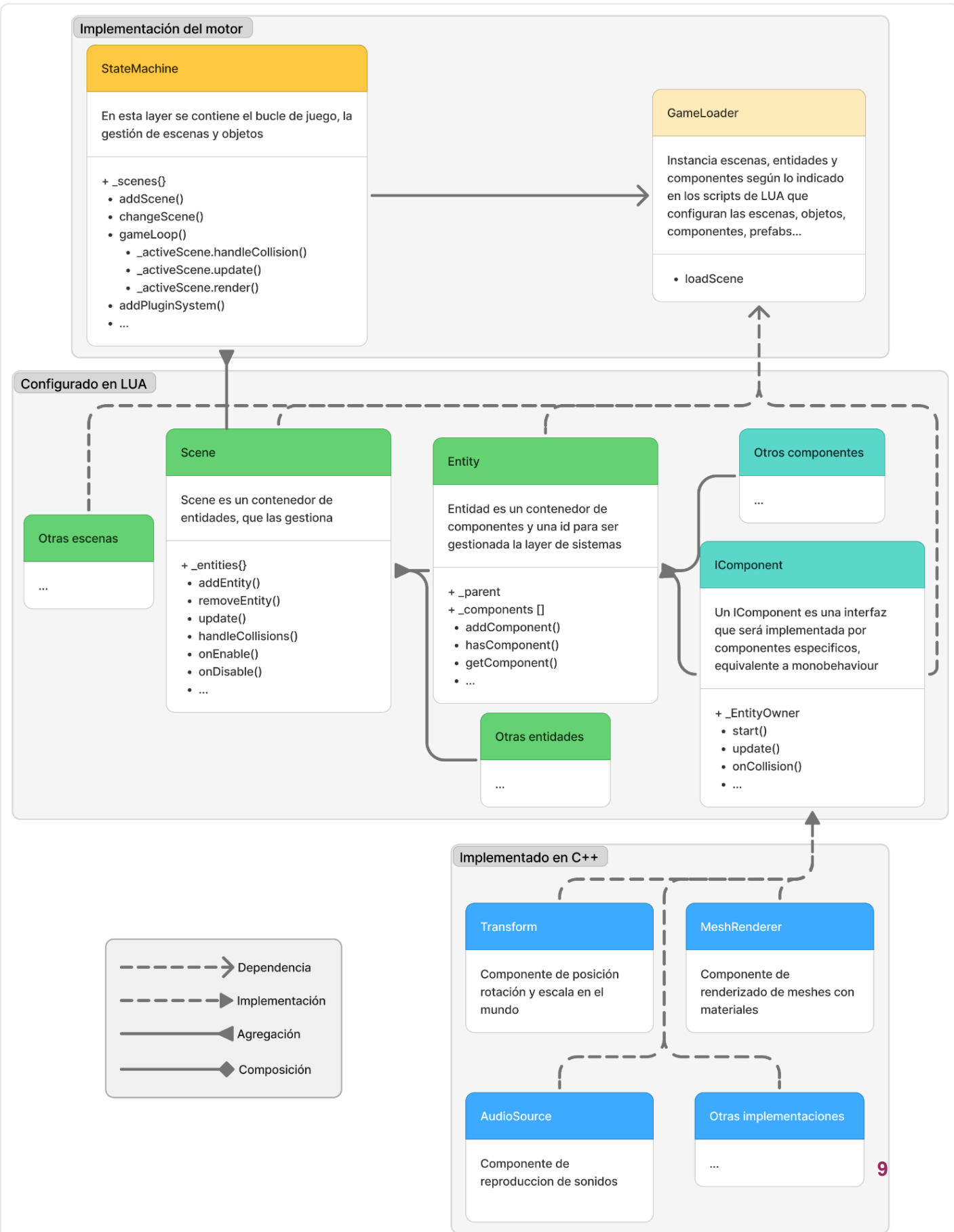
- Editor de juegos, editor de partículas, utilidades para análisis de rendimiento...

[Diagrama de clases y arquitectura del motor](#)

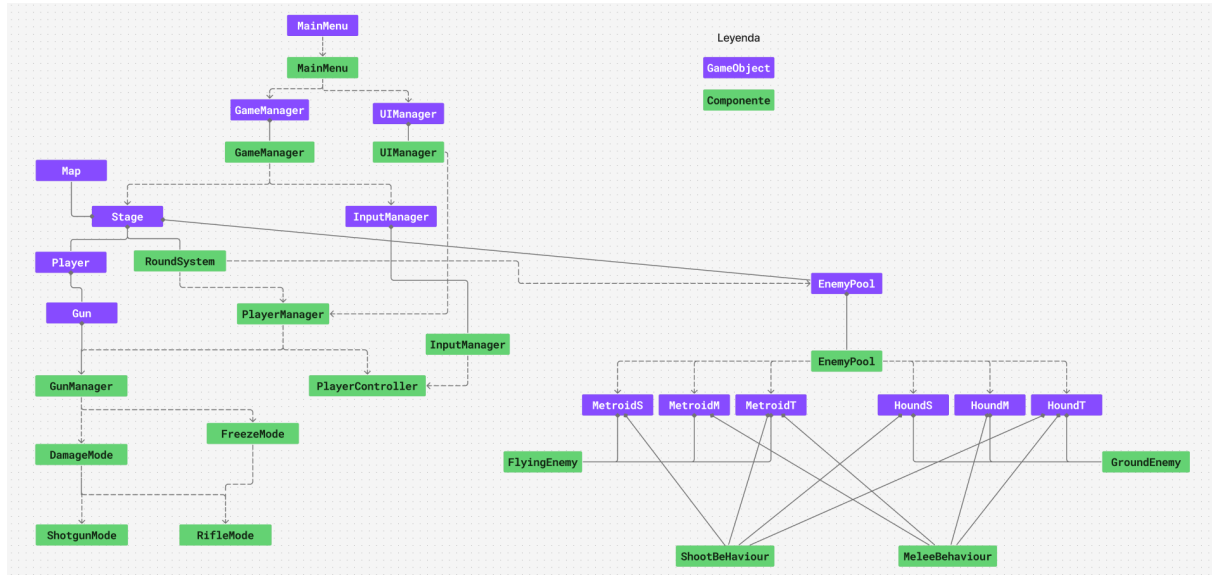
Estructura de componentes del motor

[Enlace al pdf](#)

Arquitectura de desarrollo



Estructura de componentes de MATABICHOS



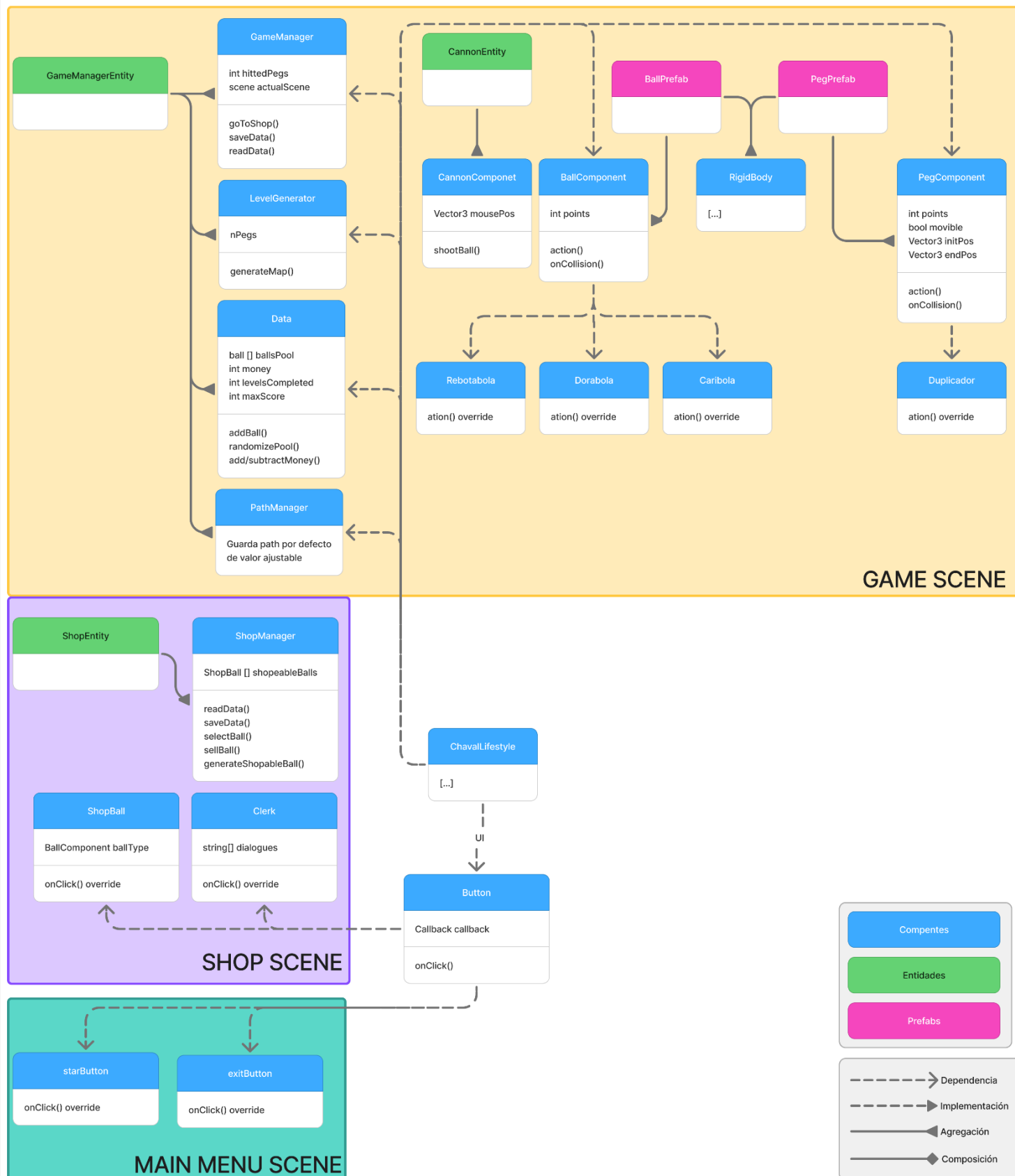
Stage contiene al mapa, al jugador y los enemigos.

Se reutilizan componentes en los enemigos para mezclar comportamientos comunes.

Estructura de componentes de TIRABOLAS

[Enlace al PDF.](#)

Section 1

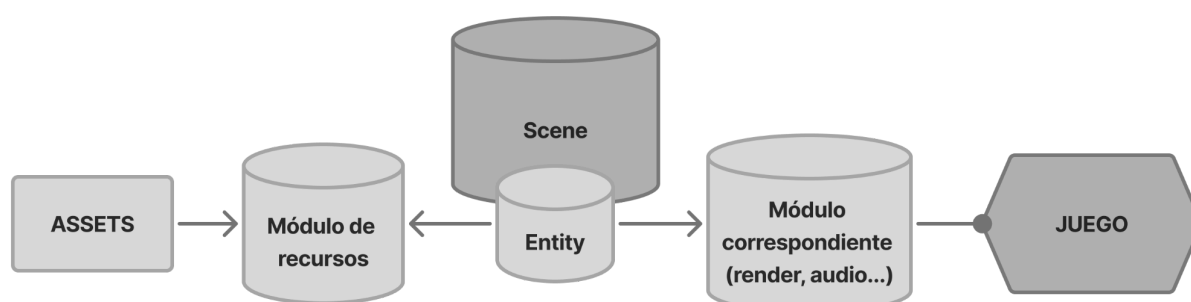


Pipeline de generación de contenido

Para desarrollar videojuegos con CHAVALES Engine el desarrollador deberá aportar *assets* para que el motor pueda trabajar con ellos. CHAVALES Engine contará con soporte para los siguientes formatos:

Formato	Uso	Módulo encargado
<i>.fbx</i>	Modelos 3D	Render
<i>.mesh</i>		
<i>.material</i>		
<i>.skeleton</i>	Animaciones modelos 3D	
<i>.png</i>	Texturas 2D	
<i>.particle</i>	Partículas	
<i>.glsl</i>	Shaders	
<i>.tff</i>	Textos UI	
<i>.ogg</i>	Música y Sonidos	Audio
<i>.lua</i>	Configuración de escenas y entidades	Core

Queda definido el flujo de datos con el esquema:



El módulo de recursos carga y almacena los assets, el desarrollador hará uso de estos en los diferentes componentes de las entidades en escena que serán gestionados por el módulo del motor correspondiente durante la ejecución del juego.

A la hora de cargar los recursos el módulo de recursos asignará un identificador único y categorizará cada asset en diferentes estructuras para la fácil gestión y acceso tanto internamente por el motor como por el desarrollador.

